

Themis

A security-intelligence platform that ingests SBOM and VEX documents, correlates them against live vulnerability feeds, governs enterprise positions, and publishes standards-based artifacts — with an AI plane that proposes and never decides.

Architecture · Workflows · Code structure · AI capability

Go 1.25 · PostgreSQL · six deployable services · no external broker

Document generated 9 August 2026 from the state of `main`

CONTENTS	1 What Themis is	6 Code structure and enforcement
	2 Architecture at a glance	7 The AI plane today
	3 How contexts collaborate	8 AI architecture: where it extends
	4 Domain model and invariants	9 Operations and configuration
	5 Workflows	

What Themis is

An organisation running software it did not entirely write needs to answer three questions, repeatedly and defensibly: *what is in our releases*, *which of it is vulnerable*, and *what did we decide to do about it*. Themis is built around the observation that these are three different kinds of question, and that conflating them is the source of most of the pain.

The first is a matter of record — an SBOM is evidence, and evidence is immutable. The second is a matter of gathering — feeds, advisories and scanners each have an opinion, and none of them is authoritative on its own. The third is a matter of judgement — whether a vulnerability matters *here* is a decision an organisation makes and must be able to justify later.

DOMAIN INVARIANT 3 – GATHERING IS NOT KNOWING

Feeds, crawlers, scanners and AI produce **Information**, which is rejectable. Only reconciliation turns Information into **Enterprise Knowledge**. Nothing that gathers may write knowledge directly.

That single rule shapes the whole system. A vendor advisory saying "not affected" does not suppress anything; it raises a proposal that a policy or a human decides. A model's recommendation is not a verdict; it is an advisory proposal recorded on `inferred` evidence that no policy is permitted to auto-accept. The record of *why* is preserved in every case, because the value of the system is not the answer — it is being able to defend the answer six months later.

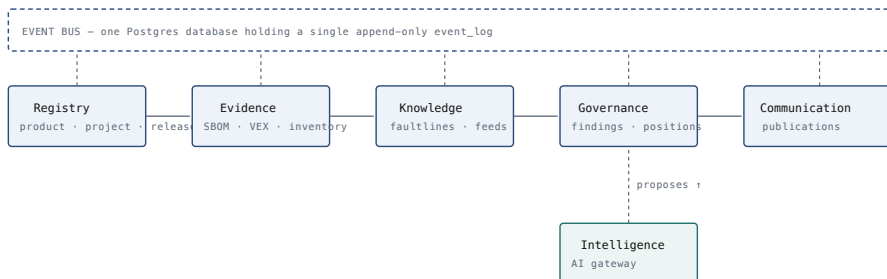
Two properties worth stating up front

Nothing is ever deleted. Source proposals are append-only. A vulnerability card is never removed, only superseded. Positions are immutable versions. A suppressed finding keeps the severity that would justify revisiting it.

The AI is optional by construction. Every correctness feature runs independently of whether the AI plane is switched on. This is not a configuration convenience; it is enforced in the design, and §8 shows where.

Architecture at a glance

Themis is six independently deployable services, each owning a bounded context, each with its own database. Four form a pipeline; one supplies identity; one sits beside the pipeline and reasons.



Solid arrows are the pipeline order. Dashed lines are the event bus, which every pipeline service publishes to and reads from. Intelligence never writes truth – it proposes into Governance and reads through HTTP read APIs.

Why a database per context

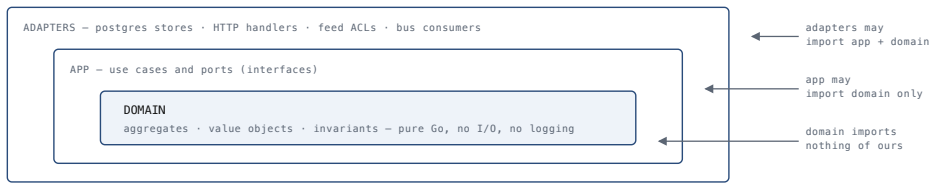
Each service owns one PostgreSQL database, and there is a seventh for the bus. This is stricter than a shared database with separate schemas, and deliberately so: a cross-context join becomes *structurally impossible* rather than merely discouraged. The boundary is enforced by the storage engine, not by review.

OBSERVED

Attempting to join `findings` (Governance) against `faultlines` (Knowledge) in one query fails with `relation "faultlines" does not exist`. The isolation is real, and it surfaces the moment someone reaches across it.

Clean architecture, applied per context

Every context is three rings with inward-only dependencies. The composition root in `cmd/<context>` wires them together and is the only place that knows about both.



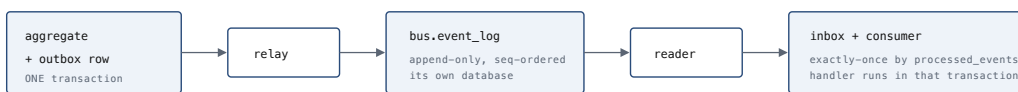
Enforced by depguard and go-cleanarch in CI, not by convention. The domain ring cannot import the logger, which is why business code cannot log.

How contexts collaborate

Contexts share nothing. They collaborate in exactly two ways, and no others.

Domain events, through a transactional outbox

A service writes its business change and its outgoing event in the *same* database transaction. A relay then moves outbox rows onto the bus. This is what makes "the finding was opened but the event was lost" impossible: both commit or neither does.



The bus is business-agnostic: it moves kernel envelopes and imports no bounded context, so a real broker can later slot in behind the same interface.

THE GAP-FREE GUARANTEE

PostgreSQL assigns the event's sequence number *before* the transaction commits, so two concurrent writers can commit out of order — seq 5 becoming visible while seq 4 is still in flight. A naive reader would consume 5, advance its cursor, and lose 4 permanently.

The reader therefore filters on `insert_xid8 < pg_snapshot_xmin(pg_current_snapshot())`: only rows whose inserting transaction is older than every still-running transaction are eligible. It cannot step over an event that will still appear.

CONSEQUENCE WORTH KNOWING

That watermark is *cluster-wide*. One long-running transaction anywhere — an idle `BEGIN` in a psql session — stalls delivery for every consumer, silently. It is why correlation's slow per-component I/O was moved outside the inbox transaction.

Read-only HTTP APIs

Where a context needs data it does not own, it calls a small client seam over the other context's read API — Knowledge reads Evidence's

inventory; Governance reads Registry's blast radius. Never a direct database read, and never a write.

Each API is **spec-first**: the OpenAPI document under `api/` is the source of truth and the Go handlers are generated from it. Hand-editing a handler is not a supported operation.

Domain model and invariants

AGGREGATE	CONTEXT	WHAT IT IS
Evidence	Evidence	An immutable, content-addressed document. Re-uploading identical bytes dedups.
Faultline	Knowledge	One enterprise card per canonical CVE, with its own opaque identity. The CVE is an alias.
Finding	Governance	One per (<i>Release, Faultline</i>) — a release-scoped concern about a vulnerability.
Position	Governance	The enterprise's decision about a Finding. Immutable versions; a new decision supersedes.
Publication	Communication	A materialized artifact — OpenVEX, CycloneDX-VEX, CSAF, report — with mutable delivery status.

The invariants that do the work

FAULTLINE

Source proposals are **append-only** for audit; the reconciled view is deterministic by source precedence. Lifecycle is forward-only — Created → Enriched → Correlated → Mature → Superseded — and a card is never deleted. A duplicate CVE arriving from a later SBOM *reuses* the existing card.

VEX OVERLAYS, NEVER DELETES

A VEX statement changes effective state; the raw finding is preserved. Vendor VEX is *gathered, not obeyed* — a reconciled `not_affected` raises a system proposal that policy or a human decides. It never auto-suppresses on its own authority.

Trust classes

Every conclusion inherits the trust of the evidence *it* depended on — tracked per field-group rather than per card, because a single vendor assertion should not drag down a verdict computed from observed ranges.

CLASS	MEANING	MAY POLICY AUTO-ACCEPT?
observed	Re-derivable fact — a version comparison, an upstream withdrawal	yes
asserted	Someone's claim — a vendor VEX statement	no
inferred	The output of non-deterministic reasoning — an AI recommendation	never

The `inferred` row is a constitutional rule, not a policy setting. No configuration makes an AI proposal auto-acceptable.

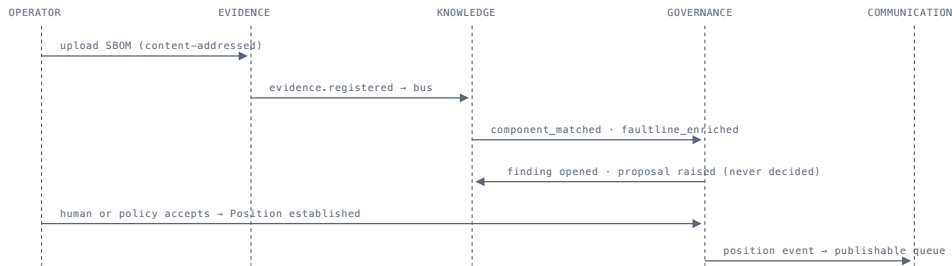
Two priority numbers, deliberately

Effective priority is how bad this is *here* — the CVE-intrinsic base score scaled by the release's blast radius. **Residual priority** is effective priority weighted by what was decided. A suppressed finding reads zero residual while keeping its effective score, so it leaves the queue without losing the severity that would justify revisiting it.

Because suppression removes work from view, a **disposition watcher** records the premise a suppressing decision rested on — the exploit signals at decision time, and an optional review date — and re-surfaces the finding when that premise drifts. It never changes the Position: it re-opens the question. An acceptance does not vanish; it expires.

Workflows

An SBOM becomes a governed posture



Every arrow crossing a lifeline is an event on the bus or a read-API call. No service reaches into another's database at any point.

Correlation, and what bounds it

Knowledge does not mirror any feed. Cards are created lazily and bounded by **relevance**: a feed enriches CVEs the estate already contains, and never imports the rest. The practical consequence is worth stating because it surprises people — on an empty database, every enabled feed enriches nothing, correctly.

A match is recorded unless the component's version is *provably out of* the reconciled, backport-aware affected range. An undecidable verdict keeps the match: a parse gap must never drop a real vulnerability.

Decision flow

Proposals accumulate on a Finding from several sources — enrichment, vendor VEX, the version-range rule, the AI. Each carries its evidence trust class. One shipped policy auto-accepts: a system-raised `not_affected` on `observed` evidence. Everything else waits for a human.

Accepting a proposal establishes an immutable Position version and emits an event, which is what makes Communication's publication queue and Intelligence's precedent index possible without either of them reading Governance's tables.

Code structure and enforcement

```

internal/
  kernel/          shared value objects + event envelop
  registry/        product → project → release, plus th
  evidence/        immutable SBOM/VEX documents, canoni
  knowledge/       faultlines, feed ACLs, reconciliatio
  governance/      findings, proposals, positions, poli
  communication/   publications and serializers
  intelligence/    the AI gateway
  platform/        observability · eventbus · inbound a

each context/
  domain/          pure Go – aggregates, value objects,
  app/             use cases and ports
  adapters/        stores, HTTP, feed ACLs, bus consume

cmd/<context>/    composition root – the only place tha
api/              OpenAPI documents; handlers are gene
docs/engineering/decisions/  EDRs – the reason of reco

```

What stops the structure eroding

The layout above is not a convention; each rule has a mechanical check that fails the build.

GATE	WHAT IT PREVENTS
depguard	Illegal imports — the logger reaching <code>domain / app</code> ; a bounded context importing the bus.
go-cleanarch	Ring violations, run once per context. It scans test files too, so a cross-layer test in the wrong directory fails.
architecture test	That <code>platform/eventbus</code> stays business-agnostic — kernel and drivers only.
coverage tiers	<code>domain / app</code> at 100%, adapters 90%, aggregate stores 80%. A package must be registered to count.
vet-tags	Rot in build-tagged code. A tagged file is invisible to the normal build and can stop compiling unnoticed.
migration reversibility	Every migration has a <code>down</code> , and up/down is a task gate.

WHY VET-TAGS EXISTS

The repository's only real-model end-to-end test had not compiled for days after a refactor renamed a seam. Nothing noticed, because no gate ever set that build tag. It type-checks every tag — integration, e2e, llm, postgres — and caught two more stale callers within the hour of being added.

COUNT	IN THE GO-FORWARD TREE
227	Go source files
170	test files
12	engineering decision records
25	database migrations
14	OpenAPI documents

The AI plane today

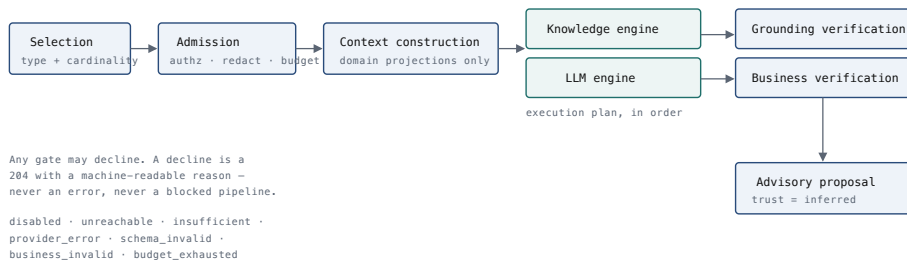
Intelligence is a reactive gateway. It holds no truth store for the business, reads everything through read APIs, and writes nothing — it returns a proposal that the calling context records as its own. It can be switched off entirely and the pipeline remains correct.

Two capabilities, split by output class

The distinction that governs everything about how a capability behaves is not what it is about, but what kind of thing it produces.

CAPABILITY	SUBJECT	OUTPUT CLASS	WHERE IT GOES
<code>recommend_position</code>	one Finding	Decision	Governance, as an advisory proposal on <code>inferred</code> evidence that no policy may accept
<code>plan_remediation</code>	one Release	Information	Nowhere. Ephemeral; proposes no stance, so there is nothing to accept

That is what makes a release-scoped capability safe to add: the worst outcome of a wrong remediation plan is a human disagreeing with it. It is also why the two take different paths through the gates below.



The Information class stops after grounding verification – no Governance stage follows it, which is precisely why that gate is treated as load-bearing there.

The two verification gates

Grounding verification, in the runtime, checks that every identifier the model cited was present in the projection it was given. It anchors to the projection, never to the shaped view the prompt rendered.

Business verification, in the consuming context, checks the claim against that context's own truth. The runtime proved the model reasoned only from the context it was handed — but that context was *supplied* to it. Only the owner of the record can confirm the claim is consistent with the system of record.

THE LIMIT OF ANY GROUNDING CHECK

Grounding verifies consistency with our record — never whether our record is right. A recommendation citing a package the flaw does not live in will pass, because the projection genuinely lists it. This is why the architecture restricts what the model is *given*, rather than trying to validate harder.

What is verified running

MEASURED ON A LIVE ESTATE, AUGUST 2026

A grounded `recommend_position` costs ~1,800 tokens and ~45 seconds on a local 20B model, returning an advisory proposal with four semantic precedents retrieved from the enterprise's own past decisions. `plan_remediation` collapses 120 findings into 12 upgrade actions, the grouping computed deterministically before the prompt. In every run, no Position was established by the AI — the constitutional rule held.

AI architecture: where it extends

The AI plane is built as a set of seams rather than a feature. Each seam is a port with a single responsibility, which is what allows capability to be added without the surrounding system changing shape. This section describes what the architecture makes possible — not a plan, but the extension points and what each one is for.

The capability is data, not code

A capability is a value in a registry. It declares its subject type and cardinality, its output class, the grounding it needs, its execution plan, the JSON Schema its raw output must satisfy, and the stances it is permitted to recommend.

```
Capability{
  ID, Version      string
  SelectionType    SelectionType // finding | rele
  Min/MaxSelection int          // cardinality, e
  Output           OutputClass  // Decision | Inf
  Needs           []ContextNeed // which projecti
  Plan            ExecutionPlan // ordered engine
  OutputSchema    string        // stage-1 struct
  AllowedStances  []Stance      // stage-2 busine
  Routing          RoutingRequirements
}
```

Adding a capability is therefore a matter of declaring one — the gateway's pipeline, validation, admission, telemetry and decline semantics all apply to it without modification. The output class is the load-bearing field: it determines whether anything reaches Governance at all, and therefore which gates are load-bearing.

The engine port

An execution plan is a list of steps, each naming an engine kind. Engines share one interface:

```
type Engine interface {
  Kind() domain.EngineKind
  Execute(ctx context.Context, in ExecInput) (EngineR
}
```

Two kinds exist. `knowledge` retrieves semantically similar past Enterprise Positions and enriches the grounding — it decides nothing, because precedent is context. `llm` reasons over the enriched grounding. A step that cannot decide defers to the next; a step that decides short-circuits the plan.

The significance is that a new kind of reasoning — a different retrieval strategy, an external reasoning service, a specialised classifier — enters as an engine behind the same port and appears in a plan as one more step. Nothing else in the gateway is aware of it.

A DELIBERATE ABSENCE

The deterministic version-range verdict is *not* a step in any AI plan. A provable verdict is a computation, not a reasoning task, so it runs in the backend — before AI and independently of whether AI is switched on. Routing it through the plan once made a correctness feature depend on an optional plane, and it was moved out.

This is the general rule the plane is organised around: **anything that can be computed is computed, and the model is asked only for what needs judgement.**

Deterministic before generative

The same principle appears at the level of individual prompts. A remediation plan groups 231 findings into a dozen package upgrades — but the grouping is a `GROUP BY` computed before the prompt, not something the model rediscovers. Asking a model to reproduce a deterministic aggregation is slow, expensive and non-repeatable; asking it which of twelve upgrades to do first is judgement.

This has a practical corollary for context: what the model is given should be the *result* of computation, not its input. A projection field that grows without bound — one affected range per rebuilt package per distribution release — can exhaust a model's budget entirely, and the failure presents as a bad model rather than an oversized prompt.

The provider port and routing

All provider-specific code is confined to `adapters/provider` behind a two-method interface. Capabilities declare *routing requirements* rather than providers, which is what allows a deployment to be pinned local-

only and a future one to route by sensitivity class without any capability changing.

The architecture anticipates routing decisions the current deployment does not need: cost-aware model selection, downgrade under budget pressure, escalation when a capability cannot determine an answer. Each is a policy over the same port. Today's single local model is a configuration of that design, not a limitation of it.

The retrieval plane

Intelligence holds one datastore — an operational semantic index of embedded past Positions. It is populated exclusively by Governance's own decision events over the bus, so the index is **derived and rebuildable**, never truth. Intelligence still owns no record.

The architectural consequence is that the AI's memory is exactly the organisation's own decision history: not a corpus, not training data, and not an opinion imported from elsewhere. Retrieval surfaces "we decided this before, here, for this reason" — and the number of precedents that grounded a recommendation is reported back to the caller, so the contribution is visible rather than asserted.

Admission is a policy seam, not a filter

Every invocation passes an admission step that authorizes the caller, redacts secrets and PII from the prompt, enforces the spend ceiling, and marks the routing class. Each is pluggable. The current deployment is local-only, which satisfies the strongest privacy rule by default; the same step is where data classification and provider clearance attach when a non-local destination exists.

WHY EVERY DECLINE IS A 204 WITH A REASON

Disabled, unreachable, declining, refused for grounding, out of budget — these demand completely different operator responses, and a bare "no proposal" cannot distinguish them. Making the reason machine-readable is what allows the AI to fail in any of these ways without a human misreading it as the model being wrong.

What the shape enables

- **Capabilities are additive.** A new subject type, a new output class, a new prompt — all declared, all inheriting the gates.

- **Reasoning strategies are substitutable.** Engines behind one port; plans are ordered lists.
- **Providers are replaceable.** Nothing outside the provider adapter knows which model answered.
- **Autonomy is a scheduling question, not a redesign.** The reactive gateway already produces proposals nothing auto-accepts; an autonomous engine spending from a capped pool proposes into the same intake.
- **The plane can be removed.** Switching it off leaves the pipeline correct — which is the property that made every one of the above safe to build.

Operations and configuration

Six binaries, one PostgreSQL server, no broker and no runtime dependency beyond the database. The AI plane additionally needs a model runtime, and only when it is enabled.

SERVICE	PORT	DATABASE
Evidence	8081	evidence
Registry	8082	evidence (co-located schema)
Governance	8083	governance
Communication	8084	communication
Knowledge	8085	knowledge
Intelligence	8086	intelligence (optional index)
— bus —	—	bus
— auth —	—	auth

The switches that change behaviour

Configuration is self-documented in a shipped example file; secrets are referenced, never inlined. Four variables change what the system *does* rather than how it is tuned:

VARIABLE	SET	UNSET
THEMIS_BUS_DATABASE_DSN	Events cross contexts — the pipeline works end to end	Log-only publisher, readers disabled; an SBOM never propagates
THEMIS_AUTH_DATABASE_DSN	Inbound /api/v1 requires an API key	Auth disabled (the node says so); THEMIS_AUTH_REQUIRED= refuses to start instead
THEMIS_<FEED>_ENABLED	That feed enriches existing cards	No outbound calls at all — the default
THEMIS_DATABASE_DSN (Intelligence)	Semantic precedent retrieval is active	Gateway is stateless; the knowledge step skips

Observability

Every node emits three signals from one shared package: structured console logs, OpenTelemetry logs and traces, and Prometheus metrics. Traces carry the same correlation id the logs carry, which is what makes a trace and its logs joinable. Spans are named by route pattern, never by concrete path.

Metrics are *scraped* while traces are *pushed*, deliberately: a counter survives having no collector, a span does not. That asymmetry costs one exporter dependency instead of two and keeps metrics useful on an isolated node.

FEEDS ARE RELEVANCE-BOUNDED

Every feed enriches CVEs the estate already contains and never mirrors a catalogue. This keeps the data proportional to the estate rather than to the industry's churn — and means enrichment can only ever follow ingestion, which is worth knowing when a fresh system looks idle.

END

Themis · Phase-3 greenfield architecture reference

Generated 9 August 2026 · reflects `main` at time of writing

Decision records: `docs/engineering/decisions/` · conventions:

`docs/engineering/CONVENTIONS.md`